

Digital Image Processing with Python and OpenCV

A Comprehensive Tutorial

Dr. Jameel Ahmad

July 31, 2026

Abstract

This tutorial provides a comprehensive guide to digital image processing using Python and OpenCV. It covers fundamental concepts and practical implementations of various image processing techniques including arithmetic operations, point operations, contrast stretching, histogram equalization, thresholding, spatial filtering, frequency domain filtering, and morphological operations. Each topic includes theoretical background, practical Python code, and visual demonstrations.

Contents

1	Introduction to Digital Image Processing	3
1.1	Setup Requirements	3
2	Arithmetic Operations on Images	3
2.1	Basic Arithmetic Operations	3
2.2	Implementation in OpenCV	3
2.3	Applications	4
3	Point Operations	4
3.1	Types of Point Operations	4
3.1.1	Brightness Adjustment	4
3.1.2	Contrast Adjustment	4
3.1.3	Negative Transformation	4
3.1.4	Gamma Correction	4
3.2	Mathematical Representation	4
4	Contrast Stretching	5
4.1	Theory	5
4.2	Implementation	5
4.3	Applications	5

5	Histogram Equalization	5
5.1	Theory	5
5.2	Implementation	6
5.2.1	Global Histogram Equalization	6
5.2.2	Adaptive Histogram Equalization (CLAHE)	6
5.3	Applications	6
6	Thresholding	6
6.1	Types of Thresholding	6
6.1.1	Simple Thresholding	6
6.1.2	Adaptive Thresholding	6
6.1.3	Otsu's Thresholding	6
6.2	Applications	7
7	Spatial Filtering	7
7.1	Smoothing Filters	7
7.1.1	Mean Filter	7
7.1.2	Gaussian Filter	7
7.1.3	Median Filter	7
7.2	Sharpening Filters	7
7.2.1	Laplacian Operator	7
7.2.2	Unsharp Masking	7
7.3	Edge Detection	7
7.3.1	Sobel Operator	7
7.3.2	Canny Edge Detection	8
8	Frequency Domain Filtering	8
8.1	Fourier Transform	8
8.2	Implementation	8
8.3	Types of Frequency Domain Filters	8
9	Morphological Operations	8
9.1	Basic Operations	9
9.1.1	Erosion and Dilation	9
9.1.2	Opening and Closing	9
9.2	Advanced Operations	9
9.3	Applications	9
10	Conclusion	9
10.1	Summary of Key Points	9
10.2	Future Directions	10

Introduction to Digital Image Processing

Digital Image Processing (DIP) is the manipulation of digital images through computer algorithms. It has applications in various fields including medical imaging, computer vision, remote sensing, and multimedia.

Setup Requirements

To run the code in this tutorial, you need:

- Python 3.6 or higher
- OpenCV (cv2)
- NumPy
- Matplotlib
- SciPy
- scikit-image

Install the required packages using:

```
1 pip install opencv-python numpy matplotlib scipy scikit-image
```

Arithmetic Operations on Images

Arithmetic operations allow us to combine and manipulate images mathematically. These operations are fundamental building blocks for more complex image processing algorithms.

Basic Arithmetic Operations

The main arithmetic operations are:

- **Addition:** $C = A + B$
- **Subtraction:** $C = A - B$
- **Multiplication:** $C = A \times B$
- **Division:** $C = A/B$

Implementation in OpenCV

```
1 # Arithmetic operations in OpenCV
2 img1 = np.full((200, 200), 100, dtype=np.uint8)
3 img2 = np.full((200, 200), 50, dtype=np.uint8)
4
5 addition = cv2.add(img1, img2)
6 subtraction = cv2.subtract(img1, img2)
7 multiplication = cv2.multiply(img1, img2)
8 division = cv2.divide(img1, img2, scale=255)
```

Applications

Arithmetic operations are used in:

- Image blending and compositing
- Background subtraction
- Masking and region-of-interest operations
- Image averaging for noise reduction

Point Operations

Point operations transform each pixel independently based on its value. These operations are simple but powerful for image enhancement.

Types of Point Operations

3.1.1 Brightness Adjustment

```
1 # Increase brightness
2 brightness = cv2.add(img, 50)
3
4 # Decrease brightness
5 darkness = cv2.subtract(img, 50)
```

3.1.2 Contrast Adjustment

```
1 # Increase contrast
2 contrast = cv2.convertScaleAbs(img, alpha=1.5, beta=0)
```

3.1.3 Negative Transformation

```
1 negative = cv2.bitwise_not(img)
```

3.1.4 Gamma Correction

```
1 gamma_corrected = exposure.adjust_gamma(img, gamma=0.5)
```

Mathematical Representation

Point operations can be expressed as:

$$I_{out}(x, y) = f(I_{in}(x, y))$$

Where f is a transformation function applied to each pixel independently.

Contrast Stretching

Contrast stretching expands the range of intensity values in an image to improve contrast.

Theory

Contrast stretching maps the minimum and maximum pixel values to 0 and 255 respectively:

$$I_{out} = \frac{I_{in} - min}{max - min} \times 255$$

Implementation

```
1 # Manual contrast stretching
2 min_val = np.min(img)
3 max_val = np.max(img)
4 stretched_manual = ((img - min_val) / (max_val - min_val) * 255).astype(
    np.uint8)
5
6 # Using OpenCV
7 stretched_cv = cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX)
8
9 # Percentile stretching
10 p2, p98 = np.percentile(img, (2, 98))
11 stretched_percentile = exposure.rescale_intensity(img, in_range=(p2, p98
    ))
```

Applications

- Enhancing medical images (X-rays, MRI)
- Improving low-contrast photographs
- Preprocessing for computer vision algorithms

Histogram Equalization

Histogram equalization redistributes pixel intensity values to achieve a uniform histogram, improving contrast.

Theory

The transformation function is based on the cumulative distribution function (CDF):

$$s_k = \sum_{j=0}^k \frac{n_j}{n}$$

Where n_j is the number of pixels with intensity j , and n is the total number of pixels.

Implementation

5.2.1 Global Histogram Equalization

```
1 equalized_global = cv2.equalizeHist(img)
```

5.2.2 Adaptive Histogram Equalization (CLAHE)

```
1 clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
2 equalized_adaptive = clahe.apply(img)
```

Applications

- Medical imaging enhancement
- Improving underwater images
- Enhancing satellite imagery

Thresholding

Thresholding converts grayscale images to binary images based on pixel intensity values.

Types of Thresholding

6.1.1 Simple Thresholding

```
1 ret, thresh_binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
2 ret, thresh_binary_inv = cv2.threshold(img, 127, 255, cv2.
  THRESH_BINARY_INV)
3 ret, thresh_trunc = cv2.threshold(img, 127, 255, cv2.THRESH_TRUNC)
4 ret, thresh_tozero = cv2.threshold(img, 127, 255, cv2.THRESH_TOZERO)
```

6.1.2 Adaptive Thresholding

```
1 thresh_adaptive_mean = cv2.adaptiveThreshold(img, 255,
2       cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY, 11,
  2)
3 thresh_adaptive_gaussian = cv2.adaptiveThreshold(img, 255,
4       cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.
  THRESH_BINARY, 11, 2)
```

6.1.3 Otsu's Thresholding

```
1 ret, thresh_otsu = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.
  THRESH_OTSU)
```

Applications

- Object detection and segmentation
- Document image processing
- Medical image analysis

Spatial Filtering

Spatial filtering modifies pixel values based on their neighbors using convolution with a kernel.

Smoothing Filters

7.1.1 Mean Filter

```
1 kernel_mean = np.ones((5,5), np.float32) / 25
2 filtered_mean = cv2.filter2D(img, -1, kernel_mean)
```

7.1.2 Gaussian Filter

```
1 filtered_gaussian = cv2.GaussianBlur(img, (5,5), 1.0)
```

7.1.3 Median Filter

```
1 filtered_median = cv2.medianBlur(img, 5)
```

Sharpening Filters

7.2.1 Laplacian Operator

```
1 laplacian = cv2.Laplacian(img, cv2.CV_64F)
2 laplacian = np.uint8(np.absolute(laplacian))
```

7.2.2 Unsharp Masking

```
1 gaussian_blur = cv2.GaussianBlur(img, (5,5), 1.0)
2 unsharp = cv2.addWeighted(img, 1.5, gaussian_blur, -0.5, 0)
```

Edge Detection

7.3.1 Sobel Operator

```
1 sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
2 sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
3 sobel_magnitude = np.sqrt(sobelx**2 + sobely**2)
```

7.3.2 Canny Edge Detection

```
1 edges = cv2.Canny(img, 100, 200)
```

Frequency Domain Filtering

Frequency domain filtering transforms images to the frequency domain, applies filters, and transforms back.

Fourier Transform

The 2D Fourier Transform:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(ux/M + vy/N)}$$

Implementation

```
1 # Compute FFT
2 f_transform = np.fft.fft2(img)
3 f_shift = np.fft.fftshift(f_transform)
4 magnitude_spectrum = np.log(np.abs(f_shift) + 1)
5
6 # Create filters
7 rows, cols = img.shape
8 crow, ccol = rows//2, cols//2
9 lp_mask = np.zeros((rows, cols), np.uint8)
10 cv2.circle(lp_mask, (ccol, crow), radius, 1, -1)
11 hp_mask = 1 - lp_mask
12
13 # Apply filters
14 f_shift_lp = f_shift * lp_mask
15 f_ishift_lp = np.fft.ifftshift(f_shift_lp)
16 img_lp = np.fft.ifft2(f_ishift_lp)
17 img_lp = np.real(img_lp).astype(np.uint8)
```

Types of Frequency Domain Filters

- **Low-pass filters:** Remove high-frequency components (smoothing)
- **High-pass filters:** Remove low-frequency components (sharpening)
- **Band-pass filters:** Pass frequencies within a specific range
- **Ideal filters:** Sharp cutoff at specified frequency
- **Gaussian filters:** Smooth transition in frequency domain

Morphological Operations

Morphological operations process images based on shapes using structuring elements.

Basic Operations

9.1.1 Erosion and Dilation

```
1 kernel = np.ones((5,5), np.uint8)
2 erosion = cv2.erode(img, kernel, iterations=1)
3 dilation = cv2.dilate(img, kernel, iterations=1)
```

9.1.2 Opening and Closing

```
1 opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
2 closing = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)
```

Advanced Operations

```
1 gradient = cv2.morphologyEx(img, cv2.MORPH_GRADIENT, kernel)
2 top_hat = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel)
3 black_hat = cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, kernel)
```

Applications

- Noise removal and image cleaning
- Object boundary extraction
- Hole filling
- Connected components analysis

Conclusion

This tutorial has covered the fundamental concepts and implementations of digital image processing using Python and OpenCV. The techniques discussed form the foundation for more advanced image processing and computer vision applications.

Summary of Key Points

- Arithmetic operations provide basic image manipulation capabilities
- Point operations enable intensity transformations
- Contrast stretching and histogram equalization improve image quality
- Thresholding enables image segmentation
- Spatial filtering provides both smoothing and sharpening capabilities
- Frequency domain filtering offers precise control over image content
- Morphological operations process images based on shape analysis

Future Directions

Advanced topics for further study:

- Deep learning for image processing
- Image restoration and denoising
- Image registration and alignment
- 3D image processing
- Video processing and analysis

References

1. Gonzalez, R.C. and Woods, R.E., *Digital Image Processing*, 4th ed., Pearson, 2018.
2. Bradski, G. and Kaehler, A., *Learning OpenCV*, O'Reilly Media, 2008.
3. Szeliski, R., *Computer Vision: Algorithms and Applications*, Springer, 2010.
4. Burger, W. and Burge, M.J., *Digital Image Processing: An Algorithmic Introduction Using Java*, Springer, 2008.